Informe de Pruebas

Examen Práctico de Karate – Billetera Digital

Automatización de pruebas de regresión de API con Karate 1.5.1

Alumno: Angel Alata Perla

Fecha de ejecución: 28/10/2026

1. Presentación

1.1 Objetivo y alcance

Se automatizó la regresión de la Billetera Digital con Karate: registro de usuarios, transferencia de 450.00 y validación en el receptor y en el Backoffice, todo en una sola ejecución y con aserciones en cada paso.

- **Objetivo:** garantizar que el flujo de transferencia entre usuarios sigue funcionando ante cambios no esperados.
- **Alcance:** 2 features y 4 escenarios. Feature Regresión (Escenarios 1, 2 y 3) y Feature Flujo Administrador.
- **Data de prueba:** generada en cada ejecución con celulares aleatorios de 9 dígitos, sin depender de datos previos.
- **Herramientas:** Karate 1.5.1, Java 17, Maven, JUnit 5 e IntelliJ IDEA.

1.2 Ambiente de pruebas

Elemento	Detalle
Aplicación bajo prueba	Billetera Enterprise (API REST de microservicios)
Servidor	134.209.211.10 (puertos 4001 a 4004)
Tipo de prueba	Regresión funcional de API, automatizada end-to-end
Header de canal	x-channel: POSTMAN (usuarios) / ADMIN (Backoffice)
Framework	Karate 1.5.1 sobre JUnit 5, Java 17 y Maven
IDE	IntelliJ IDEA
Fecha de ejecución	28/10/2026

1.3 Estrategia

La prueba genera su propia data en cada ejecución, encadena los escenarios mediante variables globales y valida cada respuesta con aserciones de estado HTTP, estructura (fuzzy matching) y reglas de negocio como el cuadre de saldos.

Microservicio	URL base	Uso en la prueba
Auth	http://134.209.211.10:4001/auth/v1	Registro y login
Wallet	http://134.209.211.10:4002/wallet/v1	Saldo, recarga, categorías, historial
Transaction	http://134.209.211.10:4003/transaction/v1	Transferencia
Admin	http://134.209.211.10:4004/admin/v1	Auditoría de transacciones y usuarios

2. Estructura de la prueba

2.1 Estructura del proyecto

Archivo / carpeta	Responsabilidad
karate-config.js	URLs por microservicio, header x-channel, data base y credenciales configurables
features/regresion.feature	Escenarios 1, 2 y 3 del flujo de regresión
features/flujo-administrador.feature	Feature 3: validación de la operación desde el Backoffice
helpers/*.feature	Registro, login, consulta de saldo y recarga reutilizables
utils/generar-celular.js	Función que genera celulares aleatorios (9 dígitos, inicia en 9)
utils/crear-usuario.js	Construye la data de prueba de cada usuario
utils/VariablesGlobales.java	Variables globales compartidas entre escenarios y features
BilleteraTest.java	Runner JUnit 5: ejecuta todo en orden y en un solo hilo

2.2 Casos de prueba

ID	Caso de prueba	Resultado esperado
CP-01	Registrar usuarios Emisor y Receptor con celular aleatorio y asignarles saldo	Registro 201 con estado ACTIVE; saldo final = saldo anterior + recarga; falla si el usuario ya existe
CP-02	Transferir 450.00 del Emisor al Receptor con la nota "Pago de Servicios"	Respuesta 200 COMPLETED con transactionId y referenceCode; saldo Emisor disminuye 450.00
CP-03	Validar la recepción en el Receptor	Saldo Receptor aumenta 450.00 y su historial tiene el movimiento INCOME con el mismo código
CP-04	Validar la operación desde el Backoffice (Administrador)	La operación figura en la auditoría con monto 450 para ambos usuarios y los saldos coinciden

3. Resultados de la ejecución

3.1 Resumen

La suite completa se ejecutó en una sola corrida desde IntelliJ IDEA y terminó sin fallos: 2 features y 4 escenarios exitosos.

Indicador	Resultado
Fecha y hora de ejecución	28/10/2026, 22:38:58
Features ejecutados	2
Escenarios ejecutados	4
Escenarios exitosos	4
Escenarios fallidos	0
Tiempo total	12.8 s (Regresión 3,974 ms + Administrador 8,866 ms)
Resultado	Exitoso: 2 features y 4 escenarios en verde

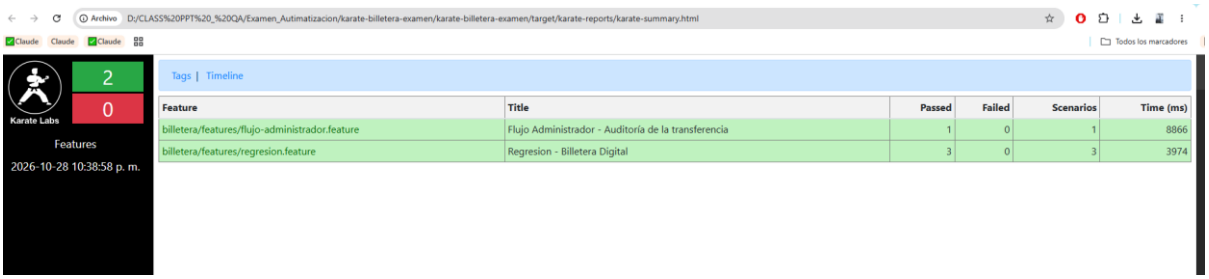


Figura 1. Karate Summary Report: 2 features pasados, 0 fallidos.

3.2 Resultado por caso de prueba

Cada escenario valida su resultado con aserciones; si una falla, el escenario se marca como fallido en el reporte.

Caso	Validaciones clave	Resultado
CP-01 - Registro de usuarios Emisor y Receptor	Registro 201 con usuario ACTIVE; error explícito si el celular ya existe (409 AUTH_006); saldo nuevo = saldo anterior + recarga; saldo consultado = saldo de la recarga	**PASSED**
CP-02 - Transferencia del Emisor al Receptor	Login con token; transferencia 200 con status COMPLETED, transactionId y referenceCode REF-…; saldo Emisor = saldo inicial - 450.00	**PASSED**
CP-03 - Validación en el Receptor	Login con token; saldo Receptor = saldo inicial + 450.00; en su historial existe 1 movimiento INCOME con el mismo referenceCode y monto 450	**PASSED**
CP-04 - Flujo Administrador	Login admin; la operación existe en la auditoría con monto 450 para el Emisor y el Receptor; saldos finales coinciden con los de los usuarios	**PASSED**

3.3 Evidencia de la ejecución

Las evidencias corresponden al reporte HTML generado por Karate en target/karate-reports, con cada paso en verde y su tiempo de respuesta.

CP-01 y CP-02 – Registro y transferencia

Se registran Emisor y Receptor con celulares aleatorios, se les asigna saldo (1000.00 y 100.00) y se valida la recarga. Luego el Emisor transfiere 450.00 con la nota "Pago de Servicios" y se valida la disminución de su saldo.

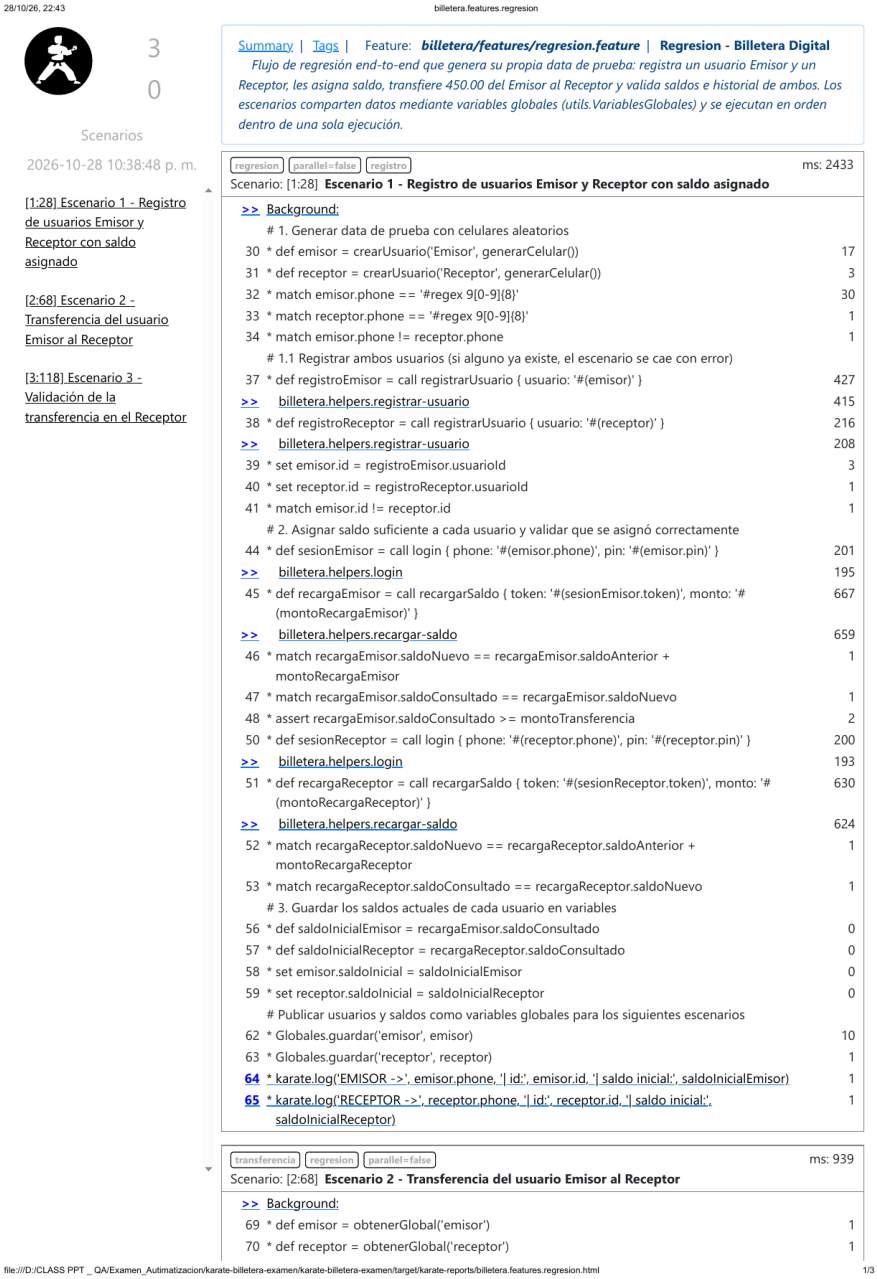


Figura 2. regresion.feature – Escenario 1 (registro y saldo) y Escenario 2 (transferencia).

CP-03 – Validación en el Receptor

El Receptor inicia sesión, se valida el incremento de 450.00 en su saldo y se encuentra en su historial el movimiento con el mismo código de transacción del Escenario 2.

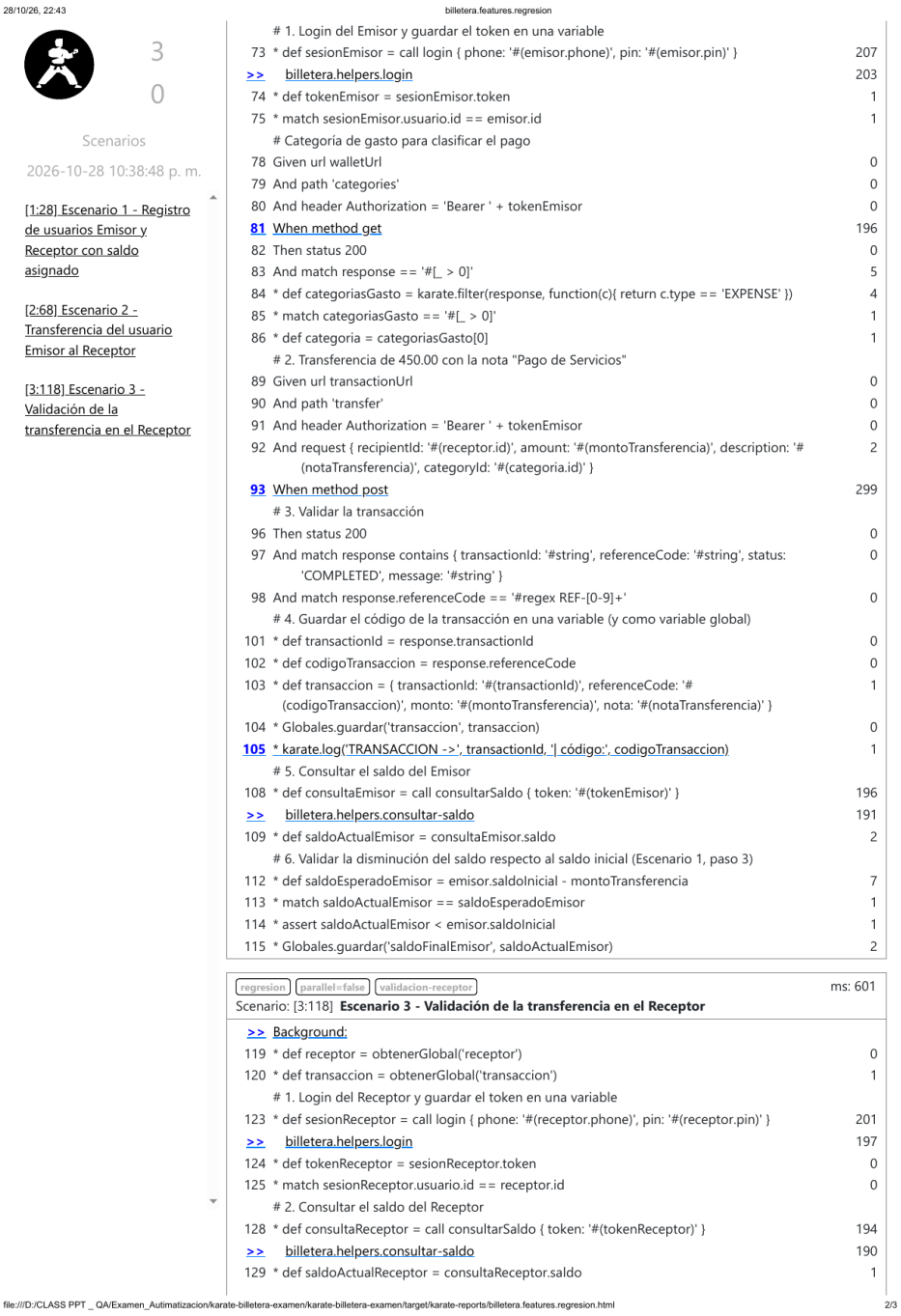


Figura 3. regresion.feature – Escenario 3.

3

0

Scenarios

2026-10-28 10:38:48 p. m.

[1:28] Escenario 1 - Registro de usuarios Emisor y Receptor con saldo asignado

[2:68] Escenario 2 - Transferencia del usuario Emisor al Receptor

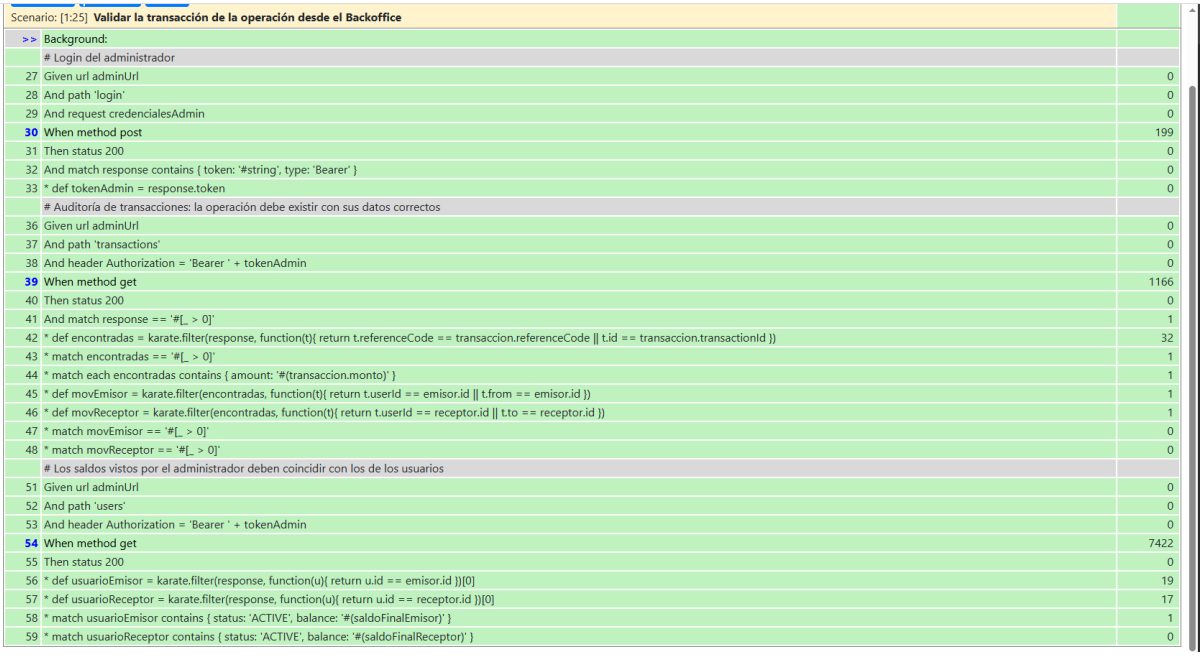
[3:118] Escenario 3 - Validación de la transferencia en el Receptor

	# 3. Validar el incremento respecto al saldo inicial (Escenario 1, paso 3)	
132	* def saldoEsperadoReceptor = receptor.saldoInicial + montoTransferencia	0
133	* match saldoActualReceptor == saldoEsperadoReceptor	0
134	* assert saldoActualReceptor > receptor.saldoInicial	0
135	* Globales.guardar('saldoFinalReceptor', saldoActualReceptor)	0
	# 4. Consultar las transacciones del Receptor	
138	Given url walletUrl	0
139	And path 'transactions'	0
140	And header Authorization = 'Bearer ' + tokenReceptor	0
141	When method get	191
142	Then status 200	0
143	And match response == '#[_ > 0]'	1
	# 5. Validar el número de transacción contra el del Escenario 2, paso 4	
146	* def movimientos = karate.filter(response, function(m){ return m.referenceCode == transaccion.referenceCode && m.type == 'INCOME' })	2
147	* match movimientos == '#[1]'	1
148	* def movimiento = movimientos[0]	0
149	* match movimiento.referenceCode == transaccion.referenceCode	0
150	* match movimiento.type == 'INCOME'	0
151	* match movimiento.amount == montoTransferencia	0

Figura 4. regresion.feature – continuación del reporte.

CP-04 – Flujo Administrador

El administrador inicia sesión, encuentra la operación en la auditoría de transacciones con el monto de 450 para Emisor y Receptor, y confirma que los saldos finales coinciden.



	Scenario: [1:25] **Validar la transacción de la operación desde el Backoffice**	
>>	Background:	
	# Login del administrador	
27	Given url adminUrl	0
28	And path 'login'	0
29	And request credencialesAdmin	0
30	When method post	199
31	Then status 200	0
32	And match response contains { token: '#string', type: 'Bearer' }	0
33	* def tokenAdmin = response.token	0
	# Auditoría de transacciones: la operación debe existir con sus datos correctos	
36	Given url adminUrl	0
37	And path 'transactions'	0
38	And header Authorization = 'Bearer ' + tokenAdmin	0
39	When method get	1166
40	Then status 200	0
41	And match response == '#[_ > 0]'	1
42	* def encontradas = karate.filter(response, function(t){ return t.referenceCode == transaccion.referenceCode || t.id == transaccion.transactionId })	32
43	* match encontradas == '#[_ > 0]'	1
44	* match each encontradas contains { amount: '#(transaccion.monto)' }	1
45	* def movEmisor = karate.filter(encontradas, function(t){ return t.userId == emisor.id || t.from == emisor.id })	1
46	* def movReceptor = karate.filter(encontradas, function(t){ return t.userId == receptor.id || t.to == receptor.id })	1
47	* match movEmisor == '#[_ > 0]'	0
48	* match movReceptor == '#[_ > 0]'	0
	# Los saldos vistos por el administrador deben coincidir con los de los usuarios	
51	Given url adminUrl	0
52	And path 'users'	0
53	And header Authorization = 'Bearer ' + tokenAdmin	0
54	When method get	7422
55	Then status 200	0
56	* def usuarioEmisor = karate.filter(response, function(u){ return u.id == emisor.id })[0]	19
57	* def usuarioReceptor = karate.filter(response, function(u){ return u.id == receptor.id })[0]	17
58	* match usuarioEmisor contains { status: 'ACTIVE', balance: '#(saldoFinalEmisor)' }	1
59	* match usuarioReceptor contains { status: 'ACTIVE', balance: '#(saldoFinalReceptor)' }	0

Figura 5. flujo-administrador.feature – todos los pasos en verde.

4. Hallazgos

Se identificaron 7 hallazgos al analizar la API, el frontend y la ejecución; los dos primeros son defectos con impacto para la seguridad o el usuario.

#	Hallazgo	Tipo	Severidad
1	El directorio GET /auth/v1/users responde sin token y expone nombre y celular de todos los usuarios (más de 3,500 registros).	Seguridad	Alta
2	La pantalla de éxito de la transferencia en el frontend muestra un código REF generado al azar en el navegador, no el referenceCode que devuelve la API.	Defecto funcional	Media
3	El endpoint de historial GET /wallet/v1/transactions no figura en la documentación; se identificó en el código del frontend y fue necesario para el Escenario 3.	Documentación	Media
4	La URL indicada (puerto 5173) es el frontend; la API real está en 4 microservicios (puertos 4001 a 4004).	Documentación	Baja
5	Todas las peticiones exigen el header x-channel; sin él la API responde 400 (GEN_001). Se usó POSTMAN para usuarios y ADMIN para el Backoffice.	Regla de negocio	Informativo
6	Las credenciales de administrador del ejemplo de la documentación (admin / superpassword) funcionaron en el servidor: son credenciales expuestas públicamente.	Seguridad	Alta
7	GET /admin/v1/transactions no devuelve id, from y to como indica la documentación, sino un movimiento por usuario con userId, type y referenceCode. La primera ejecución falló por esta diferencia y la validación se ajustó.	Documentación	Media

5. Conclusiones y recomendaciones

La automatización cubre el flujo completo de transferencia y puede repetirse en cualquier momento sin preparar datos.

- **Independencia de datos:** cada ejecución crea sus propios usuarios con celulares aleatorios, sin depender de data previa ni requerir limpieza.
- **Estado compartido controlado:** se usó un almacén de variables globales en Java y un runner de un solo hilo con @parallel=false para encadenar los escenarios en orden.
- **Validación por diferencia de saldos:** se comparan los saldos antes y después de cada operación, lo que hace la prueba robusta ante saldos iniciales distintos de cero.
- **Fallo temprano y explícito:** si un usuario ya existe o falta un dato de un escenario previo, la prueba se detiene con un mensaje claro.
- **Reutilización:** registro, login, consulta de saldo y recarga viven en features helper, dejando los escenarios legibles.
- **Triple verificación:** la transacción se confirma en la respuesta de la transferencia, en el historial del Receptor y en la auditoría del Administrador.
- **Recomendación:** corregir los hallazgos 1, 2 y 6, y actualizar la documentación de la API (hallazgos 3 y 7) antes de la siguiente versión.

6. Anexos

Entregable	Ubicación
Repositorio Git (rama main)	[pegar URL de GitHub]
Reporte PDF de Karate	Reporte_Karate_Billetera.pdf (adjunto)
Features utilizados	regresion.feature y flujo-administrador.feature (adjuntos)
Proyecto Karate	karate-billetera-examen.zip (adjunto)
Video de la ejecución	[pegar URL de Google Drive]